Assignment 1

AIM : to learn basics of OpenMP

Perquisite: C/C++, Linux basic commands

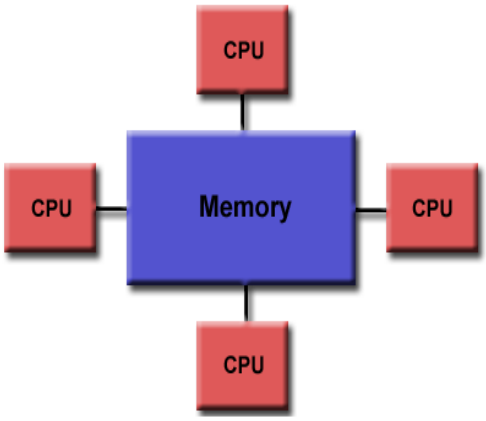
Learn basic of linux here

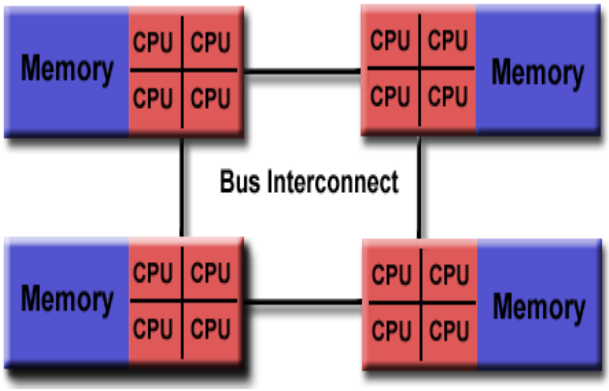
Installation : comes by default in gcc in linux OS

Install in WSL linux: sudo apt install build-essential and

sudo apt install gdb valgrind cmake

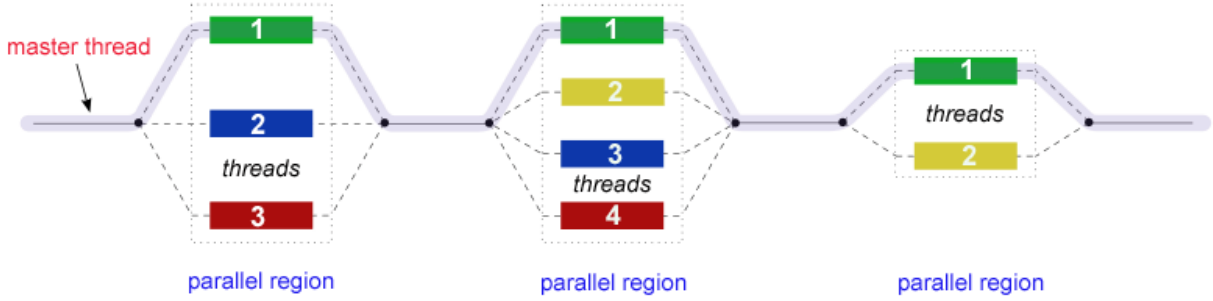
Shared Memory Model: OpenMP is designed for multi-processor/core, shared memory machines. The underlying architecture can be shared memory UMA or NUMA.





Fork - Join Model:

OpenMP uses the fork-join model of parallel execution:



All OpenMP programs begin as a single process: the **master thread**. The master thread executes sequentially until the first **parallel region** construct is encountered.

FORK: the master thread then creates a team of parallel *threads*.

The statements in the program that are enclosed by the parallel region construct are then executed in parallel among the various team threads.

JOIN: When the team threads complete the statements in the parallel region construct, they synchronize and terminate, leaving only the master thread.

The number of parallel regions and the threads that comprise them are arbitrary.

Compile OpenMP program

gcc program.c -fopenmp -o program

1. Hello world in openMP filename eg1.c:

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        printf("Hello from thread %d\n", omp_get_thread_num());
    }
    return 0;
}
gcc eg1.c -fopenmp -o eg1
./eg1
```

2. Controlling Number of Threads

```
#include <stdio.h>
#include <omp.h>
int main() {
    omp_set_num_threads(4);

    #pragma omp parallel
    {
        printf("Thread ID: %d\n", omp_get_thread_num());
    }
    return 0;
}
OMP_NUM_THREADS=8 ./eg
```

3. Parallel FOR Loop

```
#include <stdio.h>
#include <omp.h>
int main() {
```

```
    int i, sum = 0;
    #pragma omp parallel for // syntax for parallel for loop
    for (i = 1; i <= 100; i++) {
        sum += i;   // lol race condition
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

4. Fixing Race Condition with Reduction (we will study in class)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, sum = 0;
    #pragma omp parallel for reduction(+:sum)
    for (i = 1; i <= 100; i++) {
        sum += i;
    }
    printf("Correct Sum = %d\n", sum);
    return 0;
}
```

5. Measuring Execution Time

```
#include <stdio.h>
#include <omp.h>
int main() {
    double start, end;
    int i;
    long sum = 0;
    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 100000000; i++)
        sum += i;
    end = omp_get_wtime();
    printf("Time taken: %f seconds\n", end - start);
    return 0;
}
```

6. Scheduling in OpenMP

Types of Scheduling

- static
- dynamic
- guided

```
#pragma omp parallel for schedule(static,4)
```

Try

```
schedule(dynamic,2)
```

```
   schedule(guided)
7. Critical Section
#pragma omp parallel
{
    #pragma omp critical
    {
        printf("Thread %d in critical section\n",
               omp_get_thread_num());
    }
}
8.Private vs Shared Variables : Data scoping
int x = 10;
#pragma omp parallel private(x)
{
    x = omp_get_thread_num();
    printf("Thread %d: x = %d\n",
           omp_get_thread_num(), x);
}
```

Directive	Purpose
parallel	Create threads
for	Parallel loop
reduction	Avoid race condition
critical	Mutual exclusion
private	Thread-local variable
schedule	Load balancing

Questions: Experiment assignment

Q1. **DAXPY Loop:** D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^16 each, P stands for Plus. The operation to be completed in one iteration is X[i] = a*X[i] + Y[i]. Your task is to compare the speedup (in execution time) gained by increasing the number of threads. Start from a 2 thread implementation. How many threads give the max speedup? What happens if no. of threads are increased beyond this point? Why?

Q2. **Matrix Multiply :** Build a parallel implementation of multiplication of large matrices (Eg. size could be 1000x1000). Repeat the experiment from the previous question for this implementation. Think about how to partition the work amongst the threads – which elements of the product array will be calculated by each thread? Implement 2 version of parallel implementation for given task. First, use 1D threading(i.e., make a single loop run in parallel). Second, use 2D threading (i.e., use nested looping to the most suitable loops to be parallelized)

Q3. Calculation of π: This is the first example where many threads will cooperate to calculate a computational value. The task of this program will be arrive at an approximate value of π. The value of π is given by the following integral:

$$\pi = \int_0^1 \frac{4.0}{1+x^2}\,dx$$

This can be approximated as the sum of the areas of rectangles under the curve.

Sample pseudocode:

```
static long num_steps = 100000;
double step;
void main ()
{
  int i;
  double x, pi, sum = 0.0;
  step = 1.0/(double)num_steps;
  for (i=0; i<num_steps; i++) {
    x = (i+0.5)*step;
    sum = sum + 4.0/(1.0+x*x);
  }
  pi = step * sum;
}
```

Note : Submission on your github account.
Create a Master folder “UCS645” followed by subflder name LAB1, LAB2 so on.

For each lab the due date would be next week lab schedule.